43. AI'nın Ürettiği Kod Güvenli mi? Gizli Güvenlik Açıkları

Yapay Zekanın Koda Gizlice Enjekte Ettiği Güvenlik Açıklarını Avlamak
Bölüm 8 • Zorluk Derecesi: İleri Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Stanford Üniversitesi araştırmasına göre yapay zeka kullanan yazılımcıların daha çok güvenlik açığı ürettiği ve kendi kodlarının 'güvenli' olduğuna daha çok inandığı doğru mu?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Yapay zeka modelleri 'çalışan' kod üretmeye odaklıdır; 'güvenli' kod üretmek sizin şartnamelerinize bağlıdır. - 📌 En yaygın AI güvenlik kusurları: Hardcoded secret'lar, zayıf şifreleme algoritmaları ve girdi doğrulaması yapılmamış parametrelerdir. - 📌 Güvenlik bir eklenti değil; AI'a verilen promptların ve CI pipeline'ındaki SAST (Statik Güvenlik Analizi) araçlarının ayrılmaz parçası olmalıdır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir çilingir çağırdığınızı ve kapınıza kilit taktırmak istediğinizi düşünün.

Çilingir kapıya harika görünen, parlak pirinçten bir kilit takıyor. Anahtarı çeviriyorsunuz: *"Tık!"* diye dönüyor, kapı kapanıyor. Çilingir gülümsüyor: *"Harika çalışıyor!"*

Fakat çilingir gittikten sonra fark ediyorsunuz ki kilit dilini duvardaki yuvaya bağlamamış! Dışarıdan parmağınızla hafifçe dokunduğunuzda kapı ardına kadar açılıyor!

Yapay zekanın ürettiği kontrolsüz kod tam olarak o parlak pirinç kilittir: Kod hatasız derlenir, ekranda çalışır görünür. Ancak arkasında SQL Injection, yetkilendirme açığı (IDOR) veya zayıf şifreleme gibi devasa delikler bırakabilir.

Bir mühendis olarak göreviniz, anahtarın dönmesine değil, kilidin hırsızı durdurup durdurmadığına bakmaktır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

AI Kod Denetimi Kontrol Listesi (Security Checklist):

```
┌──────────────────────────────────────────────┐
│            AI KODU GÜVENLİK SÜZGECİ          │
└──────────────────────┬───────────────────────┘
                       │
  1. GİRDİ DOĞRULAMA: Dışarıdan gelen veri sanitize edildi mi?
  2. YETKİLENDİRME (AuthZ): Kullanıcı sadece kendi verisine mi erişiyor?
  3. KRİPTOĞRAFİ: MD5, SHA1, DES gibi kırılmış algoritmalar var mı?
  4. SIZINTI KONTROLÜ: API anahtarı veya şifre koda gömülmüş mü?
  5. HATA YÖNETİMİ: Stack trace veya DB şeması istemciye sızıyor mu?
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: Kıdemli Siber Güvenlik ve Kod Güvenliği Müfettişi
Aşağıdaki Python fonksiyonu bir yapay zeka tarafından üretildi ve production'a
gönderilmek üzere.

## KOD:
```python
import hashlib
import sqlite3

def register_user(username, password):
    # AI önerisi:
    hashed_pwd = hashlib.md5(password.encode()).hexdigest()
    conn = sqlite3.connect('app.db')
    cursor = conn.cursor()
    cursor.execute(f"INSERT INTO users VALUES ('{username}', '{hashed_pwd}')")
    conn.commit()
````

GÖREV:

1. Bu koddaki 3 ölümcül güvenlik açığını açıkla (SQLi, zayıf hash, salt eksikliği).
2. Bu açıkları istismar eden (exploit) senaryoları göster.
3. Kodu Argon2id / bcrypt ve parametrik SQL kullanarak kurşun geçirmez şekilde yeniden yaz.
````

````
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```python
# Kötü / AI Üretimi Güvensiz Kod: MD5 kırılmıştır ve SQL Injection açığı vardır!
import hashlib, sqlite3

def register_user_naive(user, pwd):
    # MD5 1 saniyede kırılır; Rainbow tablosu saldırısına açıktır!
    h = hashlib.md5(pwd.encode()).hexdigest()
    # String formatlama ile SQL Injection: user = "admin', '123'); DROP TABLE users;
--"
    sqlite3.connect('app.db').cursor().execute(f"INSERT INTO users VALUES ('{user}',
'{h}')")
````

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```python
# Güvenli ve Kurumsal Standart: bcrypt + Parametrik SQL Sorgusu
import bcrypt
import sqlite3

def register_user_secure(username: str, password_plain: str) -> bool:
    # 1. Girdi Uzunluk Sınırı (DoS koruması)
    if len(password_plain) < 12 or len(password_plain) > 128:
        raise ValueError("Şifre 12 ile 128 karakter arasında olmalıdır.")

    # 2. Modern Hashleme: Rastgele Salt + Yavaşlatılmış İşlem Maliyeti (Cost factor
12)
    salt = bcrypt.gensalt(rounds=12)
    password_hash = bcrypt.hashpw(password_plain.encode("utf-8"),
salt).decode("utf-8")

    # 3. Parametrik SQL: Değişkenler asla doğrudan SQL içine gömülmez!
    with sqlite3.connect("app.db") as conn:
        cursor = conn.cursor()
        cursor.execute(
            "INSERT INTO users (username, password_hash) VALUES (?, ?)",
            (username.strip().lower(), password_hash)
        )
        conn.commit()

    return True
```

💡 **Kod Analizi:** MD5 yerine yavaş ve rastgele salt üreten `bcrypt` kullanılarak Rainbow Table saldırıları imkansız kılınmış; f-string yerine `(?, ?)` parametrik sorgusu kullanılarak SQL Injection tamamen bertaraf edilmiştir.
```

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Şifre hashlemek için MD5 veya SHA-256 gibi hızlı algoritmaları kullanmak.
  - **Çözüm:** Şifreler için daima yavaş ve bellek zorlayan algoritmalar (bcrypt, Argon2id, PBKDF2) kullanın.
- **Tuzak:** SQL sorgularına değişkenleri `f'{var}'` veya `+ var` şeklinde yapıştırmak.
  - **Çözüm:** İstisnasız her SQL sorgusunda parametrik sorgu (Parameterized Queries / Prepared Statements) kullanın.
- **Tuzak:** AI'a 'bunu güvenli yap' demeden doğrudan kod yazmasını istemek.
  - **Çözüm:** Promptlarınızda 'OWASP standartlarına uy, parametrik sorgu kullan' şartını açıkça belirtin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 43: AI ve Kod Güvenliği - Gizli Açıkları Yakalamak*

- 🖥️ **Slayt 1:** Stanford Araştırması: AI kullanan geliştiriciler neden daha çok açık üretiyor?
- 🖥️ **Slayt 2:** En tehlikeli 3 kör nokta: Parametresiz SQL, MD5 şifreleme, IDOR açıkları.
- 🖥️ **Slayt 3:** bcrypt vs Argon2id: Şifre güvenliğinin matematiksel temeli.
- 🖥️ **Slayt 4:** Canlı Hack Demosu: AI'ın yazdığı giriş fonksiyonuna SQL Injection saldırısı.

Canlı Kodlama / Ekran Kaydı Senaryosu:
String birleştirme ile yazılmış bir giriş fonksiyonu açın. Kullanıcı adı yerine `' OR '1'='1` yazarak şifre bilmeden sisteme giriş yapıldığını (SQL Injection) gösterin. Ardından parametrik sorguya dönüştürüp saldırıyı engelleyin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki tüm SQL sorgularını tarayın. Herhangi bir yerde `+` veya f-string ile sorguya değişken eklenmiş mi kontrol edin ve parametrik hale getirin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Kullanıcı şifrelerini veritabanında saklarken SHA-256 veya MD5 yerine neden `bcrypt` veya `Argon2id` tercih edilmelidir? - A) bcrypt'in daha az yer kaplaması - B) SHA-256 ve MD5'in çok hızlı olması sebebiyle saniyede milyarlarca parola deneyen kaba kuvvet (Brute-Force) saldırılarına dayanamaması; bcrypt'in ise bilinçli olarak yavaşlatılmış ve 'tuzlanmış' (salted) olması - C) Sadece Python'da çalışması - D) Bilgisayarı koruması > **Doğru Yanıt:** B > **Açıklama:** Şifre hash algoritmaları saldırgan-

ların deneme hızını düşürmek için bilinçli olarak yavaş çalışacak şekilde tasarlanır; hızlı algoritmalar şifre için güvensizdir.

Soru 2: SQL Injection saldırılarını tamamen engellemenin endüstri standardı kesin çözümü nedir? - A) Kullanıcılardan tırnak işareti yazmamalarını rica etmek - B) Parametrik Sorgular (Parameterized Queries / Prepared Statements) kullanmak - C) Veritabanı şifresini değiştirmek - D) Sayfayı her gün yeniden başlatmak > **Doğru Yanıt:** B > **Açıklama:** Parametrik sorgularda veritabanı motoru kullanıcı girdisini asla çalıştırılabilir SQL kodu olarak yorumlamaz; sadece saf veri (literal string) olarak işler.